
ASSIGNMENT 4

Multivariate Linear Regression

Tutorial Number: 4
Deadline: 30.10. 2025
Max. Points: 10-15
Mandatory

Assignment

Download the archive *linRegMulti(NumPy)* from the page *Linear Regression*.

The code has the following structure:

- *data1.txt* – input data for the first part
- *data_machines.txt* – input data for the second part
- *data_machines_readme.txt* – description of individual features in the data for the second part of the task.
- *model/LinearRegression*¹ – the class implementing the whole functionality of the linear regression model (hypothesis, loss, gradient)
- *optimize/Optimizer* – Generic optimization algorithm
- *optimize/GradientDescent*¹ – gradient descent
- *utils/normalize_features*(¹) – feature scaling
- *utils/build_dict*(²) – creates a dictionary from enumerate features
- *utils/transform*(²) – transforms enumerate features into one-hot vectors
- *utils/cross_validation*() – cross-validation to use the same data for training and testing
- *visualize.py* – control visualizations similar to those in the previous assignment
- *ex4.py*¹ – the main script for the first part
- *ex4-2.py* – the main script for the second part

You need to complete classes/functions marked with ¹ in the first part, classes/functions marked with ² in the second part.

1 Multivariate Linear Regression and Feature Scaling (10 points)

Input Data

In this part you are going to predict the house prizes based on the size and the number of rooms. You can see the data distribution on Figure 1.

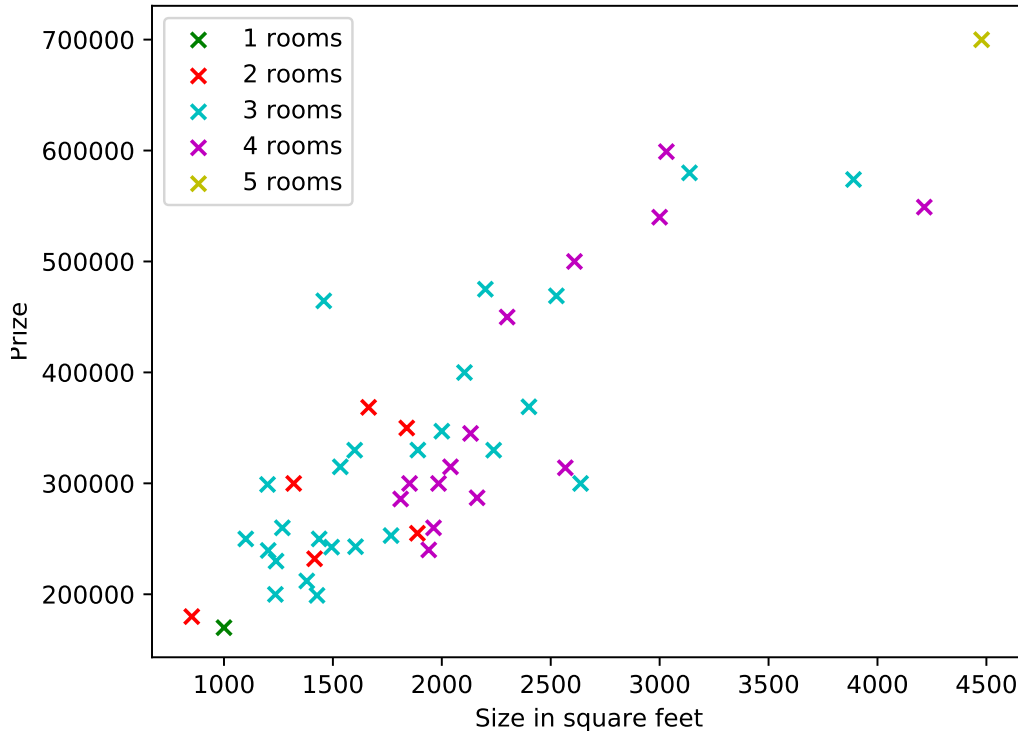


Figure 1: Input Data

Tasks

In this section you will implement a linear regression with any number of input variables. It is recommended to program the previous assignment before this one, because most tasks are only minor modifications to the tasks from the previous assignment.

1. Cost Function and Hypothesis

Implement the cost function and the hypothesis using matrix operations (without loops).

2. Gradient Descent

The gradient descent must allow multiple termination conditions to be set:

- (a) Number of iterations *num_iters*
- (b) Minimal cost *minCost*
- (c) Minimal parameters difference from previous iteration *minThetaDiff*

1-N termination conditions may be set. All specified conditions must be checked simultaneously.

3. Feature Scaling

Normalize the mean and variance of the input features .

- 4. In the file *ex4.py* add the house price prediction of 1,650 square feet and 3 rooms . Add the same prediction using **normal equation**.

5. **Tune the parameters of the gradient descent** to converge as fast as possible.

After the feature scaling the graphs should look similar to Figure 3.

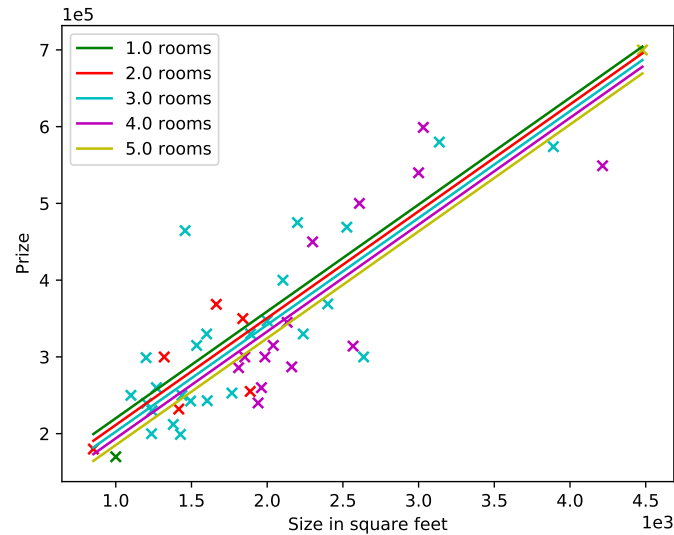


Figure 2: Linear Regression

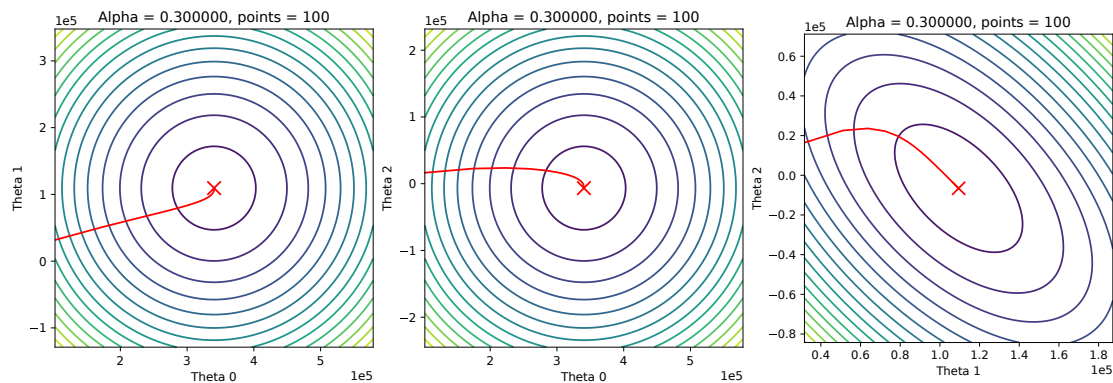


Figure 3: Loss value according to θ parameters

2 Feature Transformation (Optional, 5 points)

The goal is to predict computer performance scores based on some parameters. You can find a description of the parameters in the file *data_machines_readme.txt*. Your task is to implement a universal function to represent a text string as an enumerate feature. For this purpose, a one-hot vector is used, which is a vector of size equal to the number of all different values (plus one for an unknown value). We then represent the string with this vector, where we have only one 1 at the position corresponding to the given string and the rest of the components are zeros. Your task is to program universal functions to create this representation. In the training phase, you need to create a dictionary. The indexes in this dictionary will then correspond to the non-zero element of the one-hot vector. Your task is to complete the functions *utils/build_dict()* and *utils/transform()*.

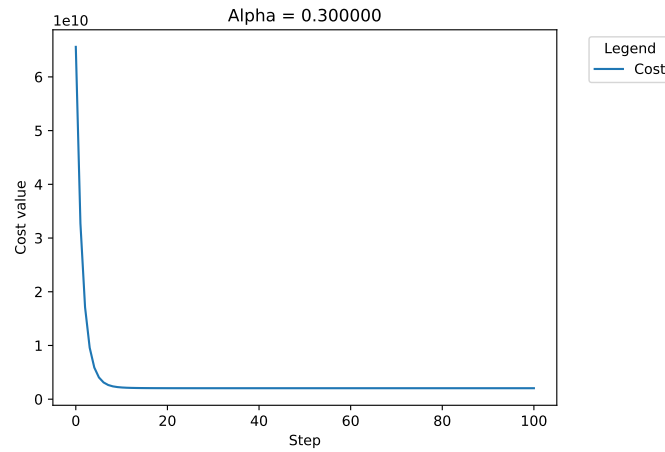


Figure 4: Convergence Graph

Example

In the training phase, we have a text:

$$\begin{pmatrix} 'first' \\ 'second' \\ 'first' \\ 'third' \\ 'first' \\ 'second' \end{pmatrix}$$

The dictionary of this text is as follows:

$$('first', 'second', 'third')$$

If we put into the transform function:

$$\begin{pmatrix} 'first' \\ 'second' \\ 'first' \\ 'fourth' \\ 'fifth' \end{pmatrix}$$

The result is this:

$$\begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

The first element of a vector corresponds to the unknown words.

The order of the elements does not matter but it needs to be consistent.